

엣지 신호처리 예산 귀속

무엇이 실시간을 막는가: 처리량 병목의 원인과 최적화 경로

2i

2026-07-29

초록

엣지 하드웨어 한 코어에서 '실시간 처리'를 약속하기 전에, 그 약속이 실제로 어느 나노초 예산 위에 서 있는지부터 재야 한다. 이 논문은 12단계로 구성된 고전적 RF 수신기 프론트엔드(동기화, 채널추정, 등화, 채널라이즈, 점유탐지, 간섭제거, 특징추출, 이상탐지, 캘리브레이션, 빔포밍)를 단일 스레드·단일 코어(Apple M3 Pro, aarch64, release 빌드)에서 실측하고, 스테이지별로 처리량 예산을 귀속시켰다. 종합처리량은 조용한 머신에서 평균 7.59 MSPS로, 40 MSPS 목표의 0.19배에 불과했다. 상위 3개 스테이지(RF 특징추출 35%, MVDR 빔포밍 29%, 적응 간섭제거 21%)가 예산의 85%를 차지했으며, 이 세 스테이지에 소스코드 판독 기반의 낙관적 구현 최적화를 전부 적용해도 종합처리량은 최대 13.23 MSPS, 즉 목표의 0.33배에 그쳤다. 결론은 명확하다 - 이 코어에서 코드 최적화만으로는 40 MSPS에 도달할 수 없고, 멀티코어(최적화 후 3~4코어, 무최적화 시 6코어) 또는 DSP/FPGA 오프로드, 또는 더 빠른 단일코어가 필요하다. 별도로 수행한 신경망 프루닝 실험은 이 정직한 회계의 필요성을 다시 확인한다 - 채널 폭을 90% 잘라 연산량(FLOPs)을 98.8% 줄여도 실제 지연은 38.5%만 줄었다. 연산량 감소가 지연 감소로 그대로 환산되지 않는다는 것 역시, 추정이 아니라 실측으로만 알 수 있는 사실이다.

목차

1 요약	3
2 왜 엡지 예산을 정직하게 재나	3
3 방법: 스테이지별 처리량 귀속과 프로파일링	3
4 측정 결과	4
4.1 종합처리량과 목표 대비 갭	4
4.2 스테이지별 예산 귀속	4
5 최적화 경로와 한계	5
5.1 상위 3개 스테이지: 무엇이 낭비고 무엇이 물리적 하한인가	5
5.2 최적화 후에도 40 MSPS는 도달하지 못한다	5
5.3 프루닝이 보여주는 것: 연산량 감소가 지연 감소로 그대로 환산되지 않는다	6
6 재현	6
7 참고문헌	7

1 요약

엡지 하드웨어 한 코어에서 “실시간 처리”를 약속하기 전에, 그 약속이 실제로 어느 나노초 예산 위에 서 있는지부터 재야 한다. 이 논문은 12단계로 구성된 고전적 RF 수신기 프론트엔드(동기화, 채널추정, 등화, 채널라이즈, 점유탐지, 간섭제거, 특징추출, 이상탐지, 캘리브레이션, 빔포밍)를 단일 스레드·단일 코어(Apple M3 Pro, aarch64, release 빌드)에서 실측하고, 스테이지별로 처리량 예산을 귀속시켰다. 종합처리량은 조용한 머신에서 평균 7.59 MSPS로, 40 MSPS 목표의 0.19배에 불과했다. 상위 3개 스테이지(RF 특징추출 35%, MVDR 빔포밍 29%, 적응 간섭제거 21%)가 예산의 85%를 차지했으며, 이 세 스테이지에 소스코드 판독 기반의 낙관적 구현 최적화를 전부 적용해도 종합처리량은 최대 13.23 MSPS, 즉 목표의 0.33배에 그쳤다. 결론은 명확하다 - 이 코어에서 코드 최적화만으로는 40 MSPS에 도달할 수 없고, 멀티코어(최적화 후 3~4코어, 무최적화 시 6코어) 또는 DSP/FPGA 오프로드, 또는 더 빠른 단일코어가 필요하다. 별도로 수행한 신경망 프루닝 실험은 이 정직한 회계의 필요성을 다시 확인한다 - 채널 폭을 90% 잘라 연산량(FLOPs)을 98.8% 줄여도 실제 지연은 38.5%만 줄었다. 연산량 감소가 지연 감소로 그대로 환산되지 않는다는 것 역시, 추정이 아니라 실측으로만 알 수 있는 사실이다.

2 왜 엡지 예산을 정직하게 재나

무선 신호처리 제품을 파는 자리에서 “실시간”이라는 단어는 값싸게 쓰인다. 초당 수십 메가샘플(MSPS)을 처리하는 스트림을 저가 엡지 코어 하나로 받아내겠다는 약속은 흔하지만, 그 약속이 실제 나노초 예산 위에 서 있는지 확인하는 작업은 드물다. 이 간극이 위험한 이유는 단순하다. 목표 처리율과 실측 처리율의 차이가 20~30% 수준이면 프로파일링과 튜닝으로 좁힐 수 있는 여지지만, 5배 이상 벌어져 있다면 코드를 아무리 다듬어도 물리적으로 도달할 수 없는 목표를 약속하고 있는 것이기 때문이다. 이 두 상황을 구분하는 유일한 방법은 스테이지별 비용을 실제로 재고, 그 비용이 알고리즘의 하한에 붙어 있는지 아니면 구현의 낭비인지를 가르는 것이다.

이 작업의 출발점은 하나의 질문이었다. 대표적인 12단계 신호처리 파이프라인 - Schmidl-Cox 동기화, Moose 주파수오프셋 추정, 최소자승(LS) 채널추정, 제로포싱(ZF) 등화, 최근접심볼 검출, 폴리페이즈 채널라이저, Urkowitz 점유탐지, NLMS 적응선형강화(간섭제거), RF지문 특징추출, 마할라노비스/코사인 스코어러, 온도/MSP 캘리브레이션, MVDR 빔포밍 - 을 저가 엡지 코어 하나에서 실행했을 때, 40 MSPS라는 목표 처리율까지의 5.3배 예산 간극은 정확히 어디에 있는가. 이 질문에 답하지 않고서는 “이 스테이지만 최적화하면 목표에 닿는다”거나 “코드 최적화로는 불가능하니 하드웨어를 바꿔야 한다” 같은 판단 중 어느 쪽도 근거를 가질 수 없다.

이 논문의 기여는 세 가지다. 첫째, 12단계 파이프라인 전체를 반복 측정해 스테이지별 예산 점유율을 순위화한다. 둘째, 상위 3개 스테이지에 대해 코드를 직접 읽어 알고리즘적으로 필수적인 하한(algorithmically-bound)과 구현이 남긴 낭비(implementation-bound)를 구분한다. 셋째, 그 구분에서 나온 재현 가능한 최적화 헤드룸을 종합처리량 예측에 대입해 “코드 최적화만으로 40 MSPS에 도달 가능한가”라는 질문에 직접, 그리고 부정적으로 답한다. 덧붙여 별도의 신경망 프루닝 실험을 통해, 이 정직한 회계 원칙이 고전적 DSP 파이프라인에만 국한되지 않는다는 것도 보여준다.

3 방법: 스테이지별 처리량 귀속과 프로파일링

측정 대상은 aarch64-apple-darwin(Apple M3 Pro, 12논리코어 중 1개만 사용), release 빌드, 단일 스레드다. 이는 목표 임베디드 ARM SoC의 대리 측정이며, 상대적 예산 배분과 자릿수 단위의 규모 추정을 위한 것이지, 상용 목표 하드웨어의 인증 수치는 아니다.

반복과 잡음 관리. 각 스테이지의 처리량은 전체 파이프라인 구축 과정을 15회 독립 재실행해(매 회차가 각 스테이지의 워밍업과 반복을 처음부터 다시 수행) 평균과 표준편차 (CV)로 함께 보고했다. 이 작업 공간은 다른 프로세스가 동시에 CPU를 쓰는 개발 머신이었기 때문에, 측정 전후로 sysctl을 통한 부하평균과 동시 실행 프로세스 수를 실측해 기록했다. 다른 무거운 빌드/연산 작업이 없는 조용한 창에서 수행한 정본 실행은 종합처리량 변동계수 (CV) 0.38%, 상위 4개 스테이지 CV 전부 1.2% 이하로 매우 안정적이었다. 반면 동시에 다른 수치연산 작업이 돌던 예비 실행에서는 종합처리량 CV가 3.0~10.2%까지, 개별 스테이지는 최대 36%까지 튀었다 - 이 값은 정본으로 채택하지 않고 참고용으로만 남겼다.

정규화 단위. 스테이지마다 한 번의 호출이 처리하는 작업 단위(샘플, 윈도우, 스냅샷 등)가 다르기 때문에, 모든 스테이지의 비용을 원표본 1개 등가당 나노초(ns/sample)로 정규화해 공정하게 비교했다. 파이프라인 종합처리량은 이 12개 값의 합의 역수($1/\sum \text{ns_per_sample}$)로 정의되며, 이는 실제로 처리량 상한을 계산하는 데 쓰이는 값과 동일하다.

순위의 강건성 검증. 서로 다른 부하 조건에서 반복 수를 15/15/15/25/20회로 바꾼 4회의 독립 실행 전부에서 상위 3개 스테이지의 집합과 순서가 동일하게 나왔다. 별도의 세션이 8회 반복으로 독립 재실행했을 때는 머신 부하가 정본 실행의 약 2배(load average 6.15 대 3.36)였는데도, 상위 3개 스테이지의 예산 점유율은 0.3%p 이내에서 유지되었고 종합처리량은 7.655

MSPS로 정본 대비 1.1% 차이에 그쳤다. 부하가 절대 처리량(초당 처리 가능한 샘플 수)에 영향을 주는 것은 예상된 일이지만, 어느 스테이지가 예산을 먹는가라는 비율은 부하에 훨씬 덜 민감했다 - 이 논문의 결론은 바로 이 상대적 귀속 위에 서 있다.

헤드룸 분석의 근거 등급. 상위 3개 스테이지에 대한 최적화 헤드룸 추정은 실제 프로파일러(perf, Instruments) 트레이스가 아니라 소스코드를 직접 읽고 계산한 아키텍처 수준 추정이다. 이 환경에서는 상승된 권한(sudo)이 필요한 프로파일러를 쓸 수 없었기 때문이다. 다만 이 추정은 실측된 두 수치와 교차검증했다 - 힙 할당 카운트와 힙 하이워터마크 델타(둘 다 전용 글로벌 얼로케이터 래퍼로 실측)가, 소스에서 직접 센 호출당 신규 Vec 개수 및 예상 버퍼 크기와 정합했다. 따라서 헤드룸의 절대 배수는 자릿수 근거로 읽어야 하지만, 그 근거가 되는 할당 패턴 자체는 추정이 아니라 실측이다.

4 측정 결과

4.1 종합처리량과 목표 대비 갭

조용한 머신에서 정본으로 실행한 데이터시트 기준 종합처리량은 **7.59 MSPS**였다. 목표 40 MSPS 대비 실시간 여유율(realtime margin)은 **0.19배**로, 최소 5.3배의 단일코어 처리량이 더 필요하다는 뜻이다. 같은 코드를 다른 시점에, 다른 잡음 조건에서 재측정한 병목귀속 정본 실행은 7.460 MSPS(변동계수 0.38%)를 얻었는데, 이는 데이터시트 수치와 1.7% 이내로 일치한다 - 서로 다른 두 실행, 다른 시각, 다른 잡음원에서 나온 값이 이만큼 가깝다는 것은 유용한 교차검증이다. 다른 동시 작업이 있는 상태에서 관측된 처리량은 낮게는 3.5 MSPS(여유율 0.09배)까지 떨어졌는데, 이는 잡음이 아니라 실측이며, 이 코어의 단일스레드 예산이 시스템 부하에 실제로 민감하다는 것을 보여주는 별도의 정직한 발견이다.

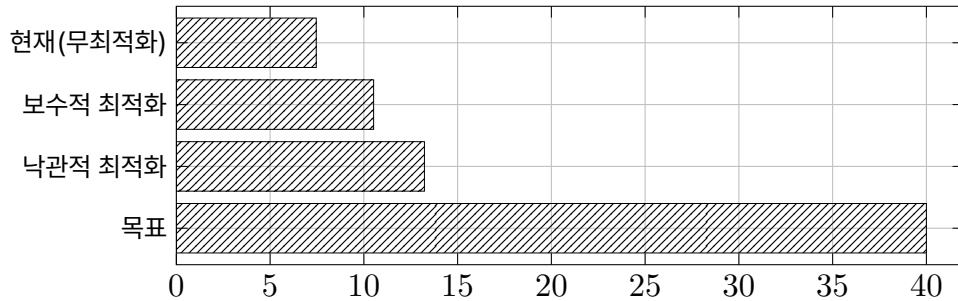


그림 1: 종합처리량: 현재 대비 최적화 시나리오 대비 목표

이 격차의 크기를 코어 수로 환산하면, 코드를 전혀 바꾸지 않고 선형으로 스케일한다는 가정 아래 40 MSPS에 도달하려면 **약 6개 코어**가 필요하다. 이 선형 스케일 가정은 이 작업에서 실측하지 않았고, 12개 스테이지 전부가 똑같이 병렬화 가능하지도 않다는 점은 4절에서 다시 다룬다.

4.2 스테이지별 예산 귀속

12개 스테이지 각각을 원표본 등가당 나노초로 정규화해 순위화한 결과는 다음과 같다 (정본 실행, 15회 반복, 변동계수 1.2% 이하).

순위	스테이지	ns/샘플	점유율	누적 점유율
1	RF 특징추출(RFFI feature extraction)	46.88	34.97%	34.97%
2	MVDR 빔포밍 가중치 갱신+적용	39.52	29.48%	64.45%
3	NLMS 적응 간섭제거(ALE)	27.93	20.83%	85.28%
4	폴리페이즈 채널라이저	7.59	5.66%	90.94%
5	최근접심볼 검출(EVM/BER)	3.91	2.91%	93.86%
6	Schmidl-Cox 타이밍 동기화	2.41	1.80%	95.65%
7	마할라노비스/코사인 스코어러	1.79	1.33%	96.99%
8	LS 채널추정	1.30	0.97%	97.96%
9	ZF 등화	1.30	0.97%	98.92%
10	Urkowitz 점유탐지	0.74	0.55%	99.47%
11	온도/MSP 캘리브레이션	0.68	0.51%	99.98%
12	Moose 주파수오프셋 추정	0.02	0.02%	100.00%

순위	스테이지	ns/샘플	점유율	누적 점유율
----	------	-------	-----	--------

상위 3개 스테이지가 예산의 **85.28%**를 차지하며, 4위(폴리페이즈 채널라이저, 5.66%)로 넘어가는 지점에서 점유율이 급격히 꺾인다. 3위와 4위의 (평균 $\pm$ 2표준편차) 구간이 전혀 겹치지 않을 정도로 이 단절은 뚜렷하다.

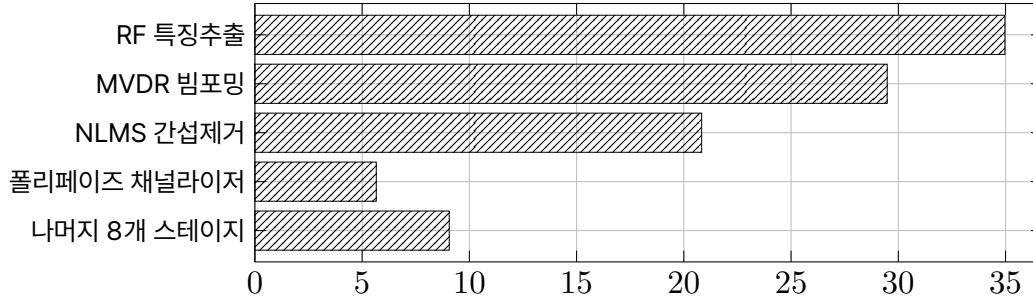


그림 2: 스테이지별 예산 점유율

이 순위는 잡음에 강건했다. 반복 수와 부하 조건이 모두 다른 4회의 독립 실행(15/15/15/ 25/20회 반복, 개별 스테이지 변동계수가 0.4%에서 10.2%까지 다양했던 조건 포함) 전부에서 상위 3개 스테이지의 집합과 순서가 동일했고, 부하가 두 배인 독립 재실행에서도 점유율은 0.3%p 이내로 유지되었다. 즉 절대 처리량은 시스템 부하에 흔들리지만, “예산을 먹는 것은 무엇인가”라는 귀속 질문의 답은 흔들리지 않았다.

5 최적화 경로와 한계

5.1 상위 3개 스테이지: 무엇이 낭비고 무엇이 물리적 하한인가

RF 특징추출(35%) - 구현 낭비가 지배적, 헤드로 클. 이 스테이지는 FFT 한 번과 25개 이상의 별도 O(W) 선형 스캔(IQ 불균형, DC 오프셋, 주파수오프셋, 엔벨로프, 피크대평균비, 위상 미분, 6개 큐물런트, 스펙트럴 특징 5종)으로 구성된다. 소스코드를 직접 인용할 수 있는 구체적 낭비가 있다 - 예를 들어 엔벨로프 신호 하나의 평균값이 표준편차·왜도·첨도 계산 내부에서 각각 독립적으로 재계산되어, 통계량 4개를 얻는 데 O(W) 스캔 9개가 든다. 위상 미분 등 다른 신호 그룹도 같은 패턴을 반복한다. 표준적인 결합모멘트 단일패스 기법(합, 제곱합, 세제곱합, 네제곱합을 한 번에 누산해 평균·분산·왜도·첨도를 대수적으로 유도하는 방법)으로 바꾸면 이 그룹들만으로도 스캔 수를 3분의 1 이하로 줄일 수 있다. 할당 카운트도 실측으로 교차검증된다 - 이 스테이지는 호출당 평균 11.7회의 힙 할당을 발생시키는데, 소스에서 직접 센 호출당 신규 버퍼 개수(10개)와 거의 일치한다. FFT 자체는 계산량이 크지 않은 $O(W \log W)$ 수준이라, 나머지 대부분은 이런 중복 계산과 할당 오버헤드다. 재현 가능한 헤드로 추정된 **약 2.0~3.0배**다.

MVDR 빔포밍(29%) - 알고리즘 하한이 지배적, 헤드로 작음. 공분산 추정과 가중치 적응은 스냅샷 수(N)와 배열 소자 수(M)에 대해 $O(N \cdot M)$ 규모이며, 이 두 항이 스테이지 비용의 대부분을 차지한다. 중요한 부정 결과가 하나 있다 - 공분산 갱신 주기를 늦춰도 스냅샷당 처리해야 하는 비용 자체는 줄지 않기 때문에, “덜 자주 갱신하라”는 이 스테이지에서는 실효적인 레버가 아니다(무시 가능한 $O(M^3)$ 역행렬 계산만 갱신 주기에 반비례해 이득을 본다). 반면 구현 낭비도 존재한다 - 8×8 정도의 작은 고정 크기 행렬을 매 호출마다 새로 힙에 할당하는 $\text{Vec} \leftarrow \text{Vec} \times \text{복소수}$ 구조가 실측 22개 안팎의 힙 할당을 발생시킨다. 이를 스택 고정배열로 바꾸면 할당은 제거되지만 $O(N \cdot M)$ 지배항 자체는 그대로 남는다. 재현 가능한 헤드로 추정된 **약 1.3~1.6배**다.

NLMS 적응 간섭제거(21%) - 알고리즘 하한이 지배적, 헤드로 작음. 표본당 비용은 필터 탭 수(order=16)에 비례하는 $O(\text{order})$ 복소 곱셈-누산이다. 이 탭 수는 좀대역 간섭을 얼마나 넓은 대역까지 추적할 수 있는지를 정하는 설계 파라미터이며, 줄이면 억제 성능이 나빠지므로 알고리즘적으로 필수적이다. 다만 실측 힙 사용량을 들여다보면, 이 스테이지의 힙 사용 대부분은 필터 상태(탭 가중치)가 아니라 매 호출 새로 할당되는 출력 버퍼 두 개(각 4096샘플, 총 128KB)다. 실제 배포에서는 캐슬러를 재사용하며 출력을 호출자 버퍼에 쓰는 형태로 이 할당을 제거할 수 있고, 탭 내적 루프는 SIMD 폭 활용 여지도 있다(단, 이 세션에서 구현·측정하지 않음). 재현 가능한 헤드로 추정된 **약 1.3~1.8배**다.

5.2 최적화 후에도 40 MSPS는 도달하지 못한다

현재 총 예산은 스테이지 12개 합인 134.05 ns/sample($=1/7.460 \times 10^6$ 초)이며, 40 MSPS를 만족하려면 이 값이 25.00 ns/sample 이하로 내려가야 한다 - 즉 현재 대비 5.36배의 개선이 필요하다. 상위 3개 스테이지에만 위 헤드를 적용하고

(하위 9개는 이미 예산의 14.72%뿐이라 손대지 않음), 보수적 시나리오와 낙관적 시나리오 두 가지로 상한과 하한을 잡았다.

시나리오	총 예산(ns/샘플)	종합처리량	목표(40 MSPS) 대비	필요 코어수
현재(무최적화)	134.07	7.46 MSPS	0.186배	6
보수적 최적화	95.06	10.52 MSPS	0.263배	4
낙관적 최적화	75.59	13.23 MSPS	0.331배	3

답은 아니었다. 상위 3개 스테이지 전부에 최적 케이스를 동시에 적용하는 낙관적 시나리오에서도 종합처리량은 목표의 0.33배에 그친다 - 여전히 최소 3배 이상 부족하다. “코드 최적화만으로 40 MSPS”는 이 코어에서 성립하지 않는다. 필요한 것은 멀티코어(최적화 후 3~4코어, 무최적화 시 6코어) 또는 그 이상의 단일코어 처리능력, 혹은 DSP/FPGA 오프로드다. 다만 선형 코어 스케일 가정은 이번 작업에서 실측하지 않았고, 12개 스테이지가 똑같이 병렬화 가능하지도 않다는 점은 정직하게 밝혀야 한다. RF 특징추출·MVDR 빔포밍·폴리페이즈 채널라이즈는 윈도우별로 독립이라 데이터 병렬화가 쉬운 편이지만, NLMS 간섭제거는 블록 사이에서 적응 상태를 스트리밍으로 이어받는 구조라 시간축을 단순히 코어별로 쪼개면 필터 연속성이 깨진다 - 코어당 파이프라인 스테이지를 분산하는 구조가 순진한 데이터 병렬화보다 안전하다. 흥미롭게도 상위 3개 스테이지는 정확히 FFT(RF 특징추출), 행렬연산(MVDR), FIR/적응필터(NLMS)라는 교과서적인 DSP 오프로드 후보이기도 하다 - 소형 DSP 코프로세서나 FPGA 패브릭이 전형적으로 특화된 연산 종류다.

5.3 프루닝이 보여주는 것: 연산량 감소가 지연 감소로 그대로 환산되지 않는다

같은 정직한 회계 원칙이 신경망 기반 컴포넌트에도 적용되는지 확인하기 위해, 별도의 소형 변조분류 신경망(1차원 CNN, 은닉 폭 64/128/128/256)에 구조적 채널 프루닝을 적용해 학습·재학습·CPU 지연을 실측했다. 이 실험은 3절의 12단계 고전 DSP 체인과는 다른 컴포넌트지만, 신호처리 프론트엔드가 학습 기반 분류기를 함께 쓸 때 흔히 마주치는 질문 - “연산량을 줄이면 실제로 빨라지는가” - 에 답한다.

설정	정확도(재학습 후)	파라미터	연산량(MACs)	지연(mean, CPU)
원본(폭 64/128/128/256)	93.91%	193,867	5,884,672	0.1181ms
50% 폭 축소	91.64% (-2.27%p)	49,835 (-74.3%)	1,500,544 (-74.5%)	0.1027ms (-13.1%)
90% 폭 축소	81.27% (-12.64%p)	2,466 (-98.7%)	68,446 (-98.8%)	0.0726ms (-38.5%)

폭을 절반으로 줄이면 연산량은 74.5% 줄지만 지연은 13.1%만 줄었고, 폭을 90% 줄이면 연산량은 98.8% 줄지만 지연은 38.5%만 줄었다. 두 축소 단계 모두 재학습 전 정확도는 일시적으로 우연 수준(9.09%)까지 붕괴했다가 재학습으로 회복되었으며, 90% 축소는 정확도를 12.6%p 회생하는 대가를 치른다. 연산량 감소율과 지연 감소율 사이의 이 큰 간극은, 이 정도로 작은 모델에서는 CPU 추론 시간이 순수 연산량이 아니라 메모리 접근과 호출 오버헤드에 더 크게 좌우된다는 뜻이다 - FLOP 카운트만 보고 최적화 효과를 예측하면 실제 지연을 과대평가하게 된다. 이는 정확히 3절과 4.1절에서 반복된 교훈과 같다. 연산의 이론적 하한이나 명세상의 감소율이 아니라, 실제로 벽시계로 잰 시간이 예산이다.

6 재현

이 논문의 스테이지별 귀속 표는 기계판독 가능한 JSON으로도 생성되며, 순위·합계·변동계수는 전부 코드가 계산한다(산문은 그 결과를 서술만 한다). 재현 명령은 다음과 같다.

병목 귀속 (15회 반복, release 빌드)

```
EDGEENCH_BOTTLENECK_REPEATS=15 cargo test --release -p edgebench --test bottlenecks -- --nocapture
```

구조적 불변식 검증 (rank 1..N 연속, 점유율 합=100%, 상위3 점유율 범위)

```
cargo test --release -p edgebench --lib -- --nocapture
```

300초 지속실행 데이터시트 (처리량/지연/메모리/누수 휴리스틱)

```
EDGEENCH_SUSTAINED_MIN_S=300 cargo test --release -p edgebench -- --nocapture
```

측정에는 새 외부 의존성을 추가하지 않았다. 힙 계측은 전용 `#[global_allocator]` 래퍼로, 프로세스 피크 상주메모리(RSS)는 `getrusage(RUSAGE_SELF)`를 직접 FFI로 호출해 얻었다. 프루닝 실험(4.3절)은 별도의 스케일링 벤치 작업에서 나온 결과이며, 동일 데이터 분할 (학습 9,900건/테스트 1,100건)로 폭 축소 비율(50%, 90%)마다 재학습을 거쳐 정확도와 CPU 지연을 함께 측정했다.

이 작업에서 명시적으로 측정하지 않은 항목도 정직하게 밝힌다 - 목표 임베디드 ARM SoC에서의 실측(개발 노트북 대리 측정만 수행), 프로파일러(perf/Instruments) 기반의 정밀 명령어 귀속(소스판독 추정으로 대체), 멀티코어 실측 스케일링(선형 가정만 사용), 그리고 4.1절에서 제안한 코드 변경 자체의 구현과 사후 재측정이다. 이 논문의 산출물은 귀속과 한계의 정직한 지도이며, 코드 변경은 그 지도 위에서 각 컴포넌트 담당자가 독립적으로 착수할 수 있는 다음 단계로 남겨둔다.

7 참고문헌

1. E. J. Kelly, "An Adaptive Detection Algorithm," IEEE Transactions on Aerospace and Electronic Systems, vol. AES-22, no. 2, pp. 115-127, 1986. (MVDR/적응 빔포밍 배경.)
2. B. Widrow and S. D. Stearns, Adaptive Signal Processing, Prentice-Hall, 1985. (NLMS 적응선형강화 배경.)
3. M. Schmidl and D. Cox, "Robust Frequency and Timing Synchronization for OFDM," IEEE Transactions on Communications, vol. 45, no. 12, pp. 1613-1621, 1997.
4. H. Urkowitz, "Energy Detection of Unknown Deterministic Signals," Proceedings of the IEEE, vol. 55, no. 4, pp. 523-531, 1967. (점유탐지/에너지검출 배경.)
5. S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," ICLR, 2016. (구조적 프루닝 배경.)
6. Rust Project, "rustfft" 및 프로젝트 자체 계측 코드베이스 (`#[global_allocator]` 래퍼, `getrusage FFI`) - 재현 명령은 5절 참조.